3. File and folder guide

Every file and folder in the repository: what it is, why it exists, and its key contents. Files marked ★ are the ones worth opening in an interview.

3.1 Repository root (the data pack, read-only input)

```
one_user_one_agent_pack/
├── README.md                Project front page: what it does, results, quick start, documentation index
├── config/                  Platform rules supplied with the pack
├── data/                    30 days of history (personal data, git-ignored)
├── replay/                  The replay day and its surprises (git-ignored)
├── pilot/                   Earlier pilot results (git-ignored)
├── simulator/               The supplied outcome simulator
├── docs/                    Data-pack guide (DATA_PACK_README.md) + reconnaissance notes written BEFORE the build
└── submission/              ★ The deliverable (everything we built)
```

Path	What it is	How the agent uses it
`README.md`	The project's front page on GitHub: what the agent does, results, quick start, documentation index, layout	Start here
`docs/DATA_PACK_README.md`	The pack's own guide: event schema, clocks (`received_at` trusted, device clocks not), duplicates, actions format, how to run the simulator	Source of truth for data formats
`config/goals.json`	The 12 goals (G01–G12): eligibility and success signals	Loaded into the `RuleBook`. G10 (null success) and G11 (`tv_app_open` never appears) are unmeasurable
`config/interventions.json`	Home-page rules (TTL 120 min, rows of 3–10, max 6 rows), push/WhatsApp rules, approved templates	Read by the planner and the verifier
`config/pricing.json`	The only prices that exist (₹1 trial, ₹149, ₹399)	The verifier rejects any other price in copy
`config/messaging_policy.md`	Consent, quiet hours, 1 message/day, honest copy, holdout, "data is not instruction", privacy	Every rule is enforced in the verifier or the database
`data/events.jsonl`	142,906 app events for ~5,000 viewers (late, duplicated, wrong clocks)	Loaded into SQLite `events`, deduplicated by `event_id`

Path	What it is	How the agent uses it
`data/profiles_snapshot.csv`	Daily warehouse export: 4,888 rows, 30 duplicated viewers, a day stale	Loaded into `profile_rows` with duplicates kept
`data/catalogue.csv`	2,014 titles in dialects d1–d6. One row (T0002) contains an injected instruction	`Catalogue`; screened by policy patterns (only T0002 matches)
`data/episodes.csv`	Episodes per content (films 1, shows 6–24, micro-dramas 20–60)	Series progress (G07), "short episodes" rows
`replay/triggers.jsonl`	2,769 app opens/logins on the replay day (90 duplicates, 400+ buffered offline)	Each one becomes a decision
`replay/live_events.jsonl`	607 other events during the day (plays, purchases, signups)	Ingested; can close plans immediately
`replay/control_events.jsonl`	Title withdrawal, G03 rule correction, model outage, data-store degradation	`AgentService.on_control`
`replay/model_faults.jsonl`	5 injected model failures (3 false claims, 1 malformed, 1 timeout)	Replayed by the model provider
`replay/arms.csv`	Randomized `agent` / `holdout` arm per viewer	Holdout gets no action, ever
`pilot/*`	Earlier pilot: arms, outcomes, and a misleading summary report	Recomputed in the impact report to show why "served vs everyone else" misleads
`simulator/simulate_outcomes.py`	Supplied simulator: plays the next 14 days given our actions	Run by `python -m oua simulate`; never modified
`docs/*.md`	Requirement matrix, data dictionary, source precedence, build plan: written before coding	Historical input; they say "BLOCKED" because they predate the build

3.2 `submission/` (the deliverable)

```
submission/
├── README.md                ★ How to install, run, test; what each output means
├── ARCHITECTURE.md          ★ Architecture note required by the brief
├── AI_COLLABORATION.md      ★ How AI tools were used, checked and sometimes rejected
├── KNOWN_LIMITATIONS.md       What is intentionally incomplete, and honest caveats
├── docs/                      These three explainer documents
├── pyproject.toml             Package definition, extras ([dev], [llm]), pytest config
├── requirements.txt           Minimal requirements (pydantic, pytest)
├── .gitignore                 Caches, SQLite files, the large simulator output
├── config/                    Runtime settings + business policy
├── src/oua/                 ★ The agent (Python package "oua" = One User, one Agent)
├── tests/                     75 tests + fixture configs
├── sample_run/              ★ Output of `python -m oua run-all`
└── viewer_state/              SQLite database (agent.sqlite), generated
```

Documents

File	Purpose
`README.md`	Setup, the one command, running with Claude, where behaviour is configured, test list, how to read a decision, layout
`ARCHITECTURE.md`	Ownership, planning, retrieval and the model, memory, source precedence, verification, latency, failure recovery, trust boundaries, measurement, scaling
`AI_COLLABORATION.md`	Which tools were used and how output was checked (replay inspection, mutation testing, simulator audit), with bugs caught and suggestions rejected
`KNOWN_LIMITATIONS.md`	Mock vs live model, provider tested against a fake API, lexical retrieval, null impact, uncalibrated confidence, scope cuts
`docs/1_HIGH_LEVEL_DESIGN.md`	The why: principles, components, decision life cycle, results, trade-offs, interview answers
`docs/2_LOW_LEVEL_DESIGN.md`	The how: tech stack, runtime model, data flow, step-by-step pipeline, algorithms, data model
`docs/3_FILE_GUIDE.md`	This file

`config/`

File	Purpose	Who would change it
★ `agent_policy.json`	**Versioned business policy**, cited in every decision (`policy_version`). It holds: • stale threshold, goal priority and labels • reviewed readings of vague goals (G05, G06, G07, G09, G12) • which goals keep their plan while open; success matchers • page sizes; row sources; per-goal interventions and follow-ups • message templates; unused templates with reasons • injection patterns; retrieval weights and `min_score` • which model tasks are on; confidence weights	Product or analytics, with review. Validated at load
`agent.json`	Runtime settings: latency budget (60 s), failure cutoff (300 s), page target (3 s), call budgets (8 model / 20 tool), timeouts, workers, load-shedding threshold, model provider/model/effort	Operations

`src/oua/` (the package, in pipeline order)

File	Lines	What it does	Key names
`__init__.py`	4	Package version and decision `SCHEMA_VERSION` (2.0.0)	
`__main__.py`	3	Lets you run `python -m oua`	
★ `check.py`	352	**Evaluation check.** Verifies the submission against the brief (section 8 requirements, the section 9 scoring areas, "what will score poorly") from the generated outputs and the pack; writes `sample_run/evaluation_check.md`	`run_checks` , `Checker` , `write_report`
`cli.py`	144	Command line: `replay` , `simulate` , `judge` , `impact` , `run-all` , `check` , `audit` ; flags `--pack --db --out --seed --workers --llm --llm-model`	`main` , `cmd_run_all` , `cmd_audit`
`settings.py`	65	Runtime settings dataclass; loads `config/agent.json` plus CLI overrides	`Settings.load`
`policy.py`	108	Loads and **validates** `agent_policy.json` . Unknown check, matcher, hero strategy or row source is an error	`Policy` , `Interpretation` , `Policy.intervention`
`timeutil.py`	41	Time helpers: parse/format ISO, IST dates, quiet-hours test, "19:00 IST on day X"	`in_quiet_hours` , `ist_at`
`pack.py`	101	Reads the pack; loads history, profiles and arms into SQLite; computes popularity; screens the catalogue	`Pack.open` , `load_pack_into_store`

File	Lines	What it does	Key names
`store.py`	583	**SQLite state.** Schema with enforced invariants; idempotent ingestion; trigger claiming; decisions, facts, actions, plans, plan history, rule versions, outcomes, lessons, control log; one-transaction context	`Store`, `tx`, `claim_trigger`, `ux_open_plan`, `ux_message_day`
`catalogue.py`	145	Titles and episodes; availability windows; withdrawal; policy-driven quarantine of instruction-like text; promotable pool; popularity ranking	`Catalogue.screen`, `promotable`, `ranked`
★ `reconcile.py`	433	**Understanding the viewer.** Facts with source/time/freshness; conflicts; dialect, status, signup, consent precedence; series progress; live-visit detection	`reconcile`, `trigger_timing`, `ViewerContext`, `Fact`, `Conflict`
★ `goals.py`	351	**Goal engine.** Versioned `RuleBook` (corrections add versions); generic eligibility checks; goal ranking; plan re-validation; success matchers	`RuleBook.correct_success`, `GoalEngine.evaluate`, `CHECKS`, `SuccessJudge`
★ `retrieval.py`	128	**The R in RAG.** BM25 index over the catalogue; per-viewer interest query from searches and plays; hits with matched terms and source events	`CatalogueIndex.search`, `interest_query`, `Hit`
★ `planner.py`	317	**Intervention planning.** Candidate rows and hero options from policy; model payload; grounding check of the model's composition; deterministic composition; follow-up message rules; copy payload and templates	`Planner.candidates`, `compose_payload`, `from_model`, `plan_message`
★ `llm.py`	316	**Model layer.** System prompt, task instructions, JSON schemas; `MockProvider` (default), `AnthropicProvider` (official SDK), fault replay; `Advisor` with three tasks and strict parsing	`AnthropicProvider.complete`, `Advisor.choose_goal`, `compose_page`, `write_copy`
★ `verifier.py`	195	**Independent gate.** Re-derives from raw rows; global, page and message checks; prices, phrases, templates, consent, quiet hours, frequency, dialect, availability	`Verifier.verify`, `Verdict`
`dependencies.py`	225	**Reliability.** Budgeted tool/model client; simulated latencies; fault windows; circuit breaker; page deadline; load shedding; virtual decision clock	`ToolClient.tool`, `ToolClient.model`, `CircuitBreaker`, `DecisionClock`
★ `agent.py`	758	**The agent.** Orchestrates one decision end to end; owns plans; builds the decision record; confidence; live events; control events (withdrawal repair, rule correction, outages); restart restore	`AgentService.handle_trigger`, `_decide`, `_compose`, `_finalise`, `on_live_event`, `on_control`
`schemas.py`	103	The decision record as a Pydantic model, with invariants (evidence resolves; PUBLISHED needs goal, actions and agent arm; no-action needs a reason)	`Decision`

File	Lines	What it does	Key names
`harness.py`	158	**Replay.** Merges triggers, live and control events by time; worker pool and per-viewer serialisation; writes decisions/actions; latency report	`timeline`, `run_replay`, `latency_report`
★ `measure.py`	497	**Closing the loop.** Attribution-aware judge for every plan; exposure; lessons; ITT impact with CIs; naive-comparison warnings; pilot recomputation; reports; per-viewer export; audit	`judge_all`, `judge_plan`, `impact_report`, `audit`, `write_final_report`

`tests/` (75 tests)

File	Tests	What it proves
`conftest.py`		Builds a **synthetic mini pack** (fake viewers, 24 generated titles, the real rule configs, one injected-instruction title) so tests never need the private data. Fixtures `mini` and `build`
`test_idempotency.py`	7	**Brief: late/duplicate triggers.** Duplicate delivery, buffered offline open, wrong device clock, history redelivery, live redelivery, crash between claim and commit, same-session reopen
`test_conflicts.py`	5	**Brief: conflicting sources.** Three dialects, duplicate profile rows, purchase vs stale "never", opt-out vs snapshot, missing profile ≠ "never"
`test_failures.py`	6	**Brief: tool failure.** Model 503 and circuit breaker, 45 s timeout (no second wait), malformed output, data store down with and without a snapshot, budget exhaustion
`test_goal_correction.py`	4	**Brief: corrected goal rule.** Only open G03 plans change; closed outcomes untouched; `window_hours` inherited; new goals and judging use the new rule; survives restart; refuses to make a goal unmeasurable
`test_safety.py`	12	Holdout, fail-closed arm, injection never promoted, unsupported model claim rejected, G10/G11 never committed, no-content dialect, compliant follow-up, verifier unit checks (quiet hours, phrases, fake price, wrong dialect, bad deeplink, consent, cap), database-level cap
`test_rag_llm.py`	9	Retrieval cites the viewer's own events, never crosses dialect or quarantine, excludes watched titles. Grounding removes invented IDs; ungrounded output falls back to rules. The model can decline but not add a message. Bad copy is replaced. Prompts carry no raw rows. Load shedding
`test_anthropic_provider.py`	4	The **real Anthropic SDK** against a local fake API: request shape (model, structured-output schema, effort, fallbacks), 529 → unavailable, slow → timeout, refusal handling
`test_policy.py`	5	Behaviour changes by config only; a brand-new goal works with config alone; free-text goals refused; interpretations editable; invalid policy fails at load
`test_measurement.py`	5	Live purchase closes the goal; pre-goal purchase not credited; attribution uses buffering, not device clock; judge and impact end to end; statistics
`test_harness.py`	4 (+6 cases)	300-open burst within budget; one viewer never decided concurrently; withdrawal repairs live pages in time order; the decision schema rejects 6 kinds of invalid records

File	Tests	What it proves
`test_real_pack.py`	4	Runs only if the pack is present: exactly T0002 flagged; real G03 correction inherits 48 h; G10/G11 unmeasurable; pilot numbers reproduce and flip under ITT
`fixtures/config/`		Copies of the pack's rule files (goals, interventions, pricing, policy text), not personal data

`sample_run/` (generated by `python -m oua run-all`)

File	What it contains	Use in an interview
★ `decisions.jsonl`	One JSON decision per trigger (2,687): goal, action or no-action reason, evidence with freshness, conflicts, assumptions, checks, model tasks, retrieval, timing	Show one record next to the brief's section 4 example
`actions.jsonl`	1,154 published actions in the simulator's format (home pages, push, WhatsApp)	Input to the simulator
`outcomes.jsonl`	Every plan (both arms) judged: succeeded/failed/inconclusive, reason, signal, exposure	"Close the loop"
`latency_report.json`	p50/p95/max, queue wait, per-hour load, per-outage-window behaviour, call maxima	Real-time reliability
`impact_report.md`	ITT result, per-goal effects, behaviour metrics, balance table, why naive comparisons mislead, pilot recomputed	"Prove impact"
`impact.json`	Same numbers as data	
★ `final_report.md`	One-page summary: what happened, latency, each surprise with example decision IDs, model faults table, outcomes, impact	Best single file to open first
`replay_summary.json`	Counts, live-event effects, control events with what each affected	
★ `evaluation_check.md`	Output of `python -m oua check --run-tests`: 48 checks against the brief, all passing, each with evidence	Show this to prove every requirement is met
`sim/simulator_report.json`	The simulator's own audit: **0 policy problems**, pages accepted/shown, messages delivered	Independent safety evidence
`sim/outcome_events.jsonl`	Simulated 14 days of viewer behaviour (git-ignored, regenerated)	
`viewer_state/U-xxxxx.json`	1,746 per-viewer exports: arm, facts at the last decision, decisions, actions, plans, plan history, outcomes, lessons	"Durable per-viewer state"

`viewer_state/`

`agent.sqlite` is the live database written by the replay (git-ignored, regenerated). Inspect it with `python -m oua audit --viewer U-02344 --at 2026-10-01T12:00:00Z`, which shows what the agent knew, decided and observed at that moment.

3.3 Suggested 10-minute walkthrough for the interview

1. **The problem and the scoring.** `docs/1_HIGH_LEVEL_DESIGN.md` sections 1.1–1.3.
2. **Run it.** `python -m oua run-all` (about 40 s), then `python -m oua check --run-tests` (48 checks against the brief, including the 75 tests).
3. **Open** `sample_run/final_report.md`**.** Surprises with decision IDs, 0 policy problems, latency.
4. **Follow one viewer.** `python -m oua audit --viewer U-02344`. Walk the decision `D-000001` in `decisions.jsonl`:
    1. the dialect conflict (d3 vs d1);
    2. the 12 goal evaluations;
    3. G07 chosen, with the model agreeing;
    4. the page and the push at 19:00 IST next day;
    5. 23 verifier checks passed;
    6. confidence 0.80 with its terms;
    7. later judged failed, and the lesson stored.
5. **Show the trust boundary.** `agent.py: _compose` → `planner.py: from_model` (grounding) → `verifier.py: verify`. Then the false model claim on U-01991 (rejected).
6. **Show config over code.** `config/agent_policy.json`: change `trigger.stale_after_s` and the behaviour changes; `tests/test_policy.py` proves a new goal needs config only.
7. **Prove impact honestly.** `sample_run/impact_report.md`: +1.0 pt with a CI spanning zero, and the pilot's "+22.4 pts" becoming −1.3 under randomized arms.
8. **Close with limitations and AI use.** `KNOWN_LIMITATIONS.md` and `AI_COLLABORATION.md`: what was caught by tests and mutation testing, and what was rejected.